피드 searchParams, 서버 async, 클라이언트 버튼

- 날짜: 2026-10-30
- 대상: Next.js 16.4.3, `lgcns-fe-next`
- 설정: `next.config.ts`의 `cacheComponents: true`, `partialPrefetching: true`
- 기준 파일은 현재 소스다. 페이지가 `searchParams`를 `<Suspense>` 안으로 넘기고, 서버 컴포넌트가 `await`한다. 버튼만 `"use client"`다.

`/feed?page=1&limit=10&sort=asc&search=cat`에서 `?` 뒤가 `searchParams`다. Next.js 15부터 이 값은 페이지 props의 프로미스다. 값은 문자열이다. 이 객체는 `URLSearchParams`가 아니므로 `.get()`은 없다.

1. 파일과 선언

파일	선언	모듈	하는 일
app/(main)/feed/page.tsx	function FeedPage	서버 컴포넌트	프로미스를 <Suspense> 자식으로 넘긴다. await가 없다
component/templates/FeedContent.tsx	async function FeedContent	서버 컴포넌트	await searchParams 후 문단을 그린다
component/molecules/FeedNextButton.tsx	"use client" + function FeedNextButton	클라이언트 컴포넌트	useRouter().push로 다음 쿼리를 연다
app/(main)/blog/[blogId]/page.tsx	function BlogPage	서버 컴포넌트	params 프로미스를 넘긴다
component/templates/BlogContent.tsx	async function BlogContent	서버 컴포넌트	await params. 클라이언트 자식이 없다
app/(main)/product/[...productCategoryName]/page.tsx	function ProductPage	서버 컴포넌트	catch-all params 프로미스를 넘긴다
component/templates/ProductContent.tsx	async function ProductContent	서버 컴포넌트	배열을 await하고 CustomTitle을 호출한다
component/atoms/CustomTitle.tsx	function CustomTitle	서버 컴포넌트	twMerge로 클래스 문자열을 합친다

(main)은 라우트 그룹이다. URL에는 없다. 피드 주소는 /feed다.

FeedContent의 async는 ECMAScript async 함수다. 호출 결과가 프로미스이고, await는 그 함수 안에서 다음 줄로 넘어가기 전에 searchParams가 풀리기를 기다린다. HTML의 <script async>와 같은 속성이 아니다.

2. 로딩 시점

이 라우트 파일은 HTML에 <script src>로 적혀 있지 않다. Next가 서버에서 페이지 함수를 실행하고, "use client" 모듈만 브라우저 번들로 보낸다.

코드	적힌 곳	로딩
<script src="js/main.js">	lgcns-fe-html/index.html, about.html 본문 끝	속성 없음. 파서가 그 지점에서 스크립트를 받아 실행한 뒤 다음 HTML로 간다
<script src="js/main.js" async>	lgcns-fe-html/contact.html	async. 받는 동안 파서는 계속 가고, 받으면 실행한다
<script type="module" src="/src/main.tsx">	lgcns-fe/index.html	type="module". 모듈 스크립트는 지연 실행된다. defer를 적어도 모듈의 기본 동작과 같다
async function FeedContent	서버 컴포넌트	브라우저 스크립트 태그가 아니다. 서버가 요청 중에 이 함수를 실행한다
"use client" FeedNextButton	클라이언트 컴포넌트	Next가 클라이언트 모듈로 보낸다. onClick은 버튼을 누를 때 브라우저에서 실행된다

"use client"는 호이스팅 문법이 아니다. 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 같은 파일의 함수를 async로 두면 React가 개발 모드에서 이렇게 알린다.

> <FeedContent> is an async Client Component. Only Server Components can be async at the moment.

현재 FeedContent.tsx에는 "use client"가 없다. async는 그 서버 함수에만 있다. FeedNextButton.tsx에는 "use client"가 있고 함수는 일반 함수다. useRouter는 그 클라이언트 함수 맨 위에서 호출한다.

3. 쿼리 샘플

페이지가 받는 타입은 page, limit, sort, search가 모두 string이다. 공식 페이지 문서의 searchParams는 키마다 string | string[] | undefined다. 같은 키가 반복되면 배열이고, 없는 키는 undefined다.

URL	await searchParams 후
/feed?page=1&limit=10&sort=asc&search=cat	page "1", limit "10", sort "asc", search "cat"
/feed?page=2&limit=10&sort=asc&search=cat	page만 "2"
/feed	네 키가 없다. 런타임 값은 undefined다. 소스 타입은 이 경우를 좁혀 적지 않았다

화면에 찍히는 문단은 `page: {page}` 형태다. `/feed?page=1&limit=10&sort=asc&search=cat`이면 네 줄이 `1`, `10`, `asc`, `cat`이다.

Next 버튼은 받은 문자열로 다음 주소를 만든다.

```
router.push(
  `/feed?page=${parseInt(page) + 1}&limit=${limit}&sort=${sort}&search=${search}`
)
```

`parseInt("1")`은 `1`이고, 거기에 `1`을 더해 쿼리는 `page=2`가 된다. 이 호출은 진법 인자를 넘기지 않는다. `"1"`처럼 `0`으로 시작하지 않는 십진 문자열은 10진으로 읽힌다. `page`가 `undefined`이면 `parseInt` 결과는 `NaN`이고, 템플릿에 넣으면 글자 `NaN`이 된다. `limit` 같은 값이 `undefined`이면 템플릿은 글자 `undefined`를 붙인다. 기본값을 넣는 코드는 이 파일에 없다.

`router.push`는 브라우저에서 앱 라우터로 그 주소를 연다. `useRouter` 문서는 프로그래밍으로 경로를 바꿀 때만 이 훅을 쓰고, 일반 이동은 `<Link>`를 쓰라고 적는다. 이 버튼은 현재 쿼리로 다음 `page`를 계산하므로 `push`를 쓴다.

4. await는 Suspense 자식 안에

`cacheComponents`가 켜지면 `searchParams`는 요청 URL이 있어야 아는 런타임 데이터다. 페이지 함수가 반환하기 전에 `await searchParams`를 하면, 그 대기는 `<Suspense>` 밖이다. 개발 서버는 그때 "Next.js encountered runtime data during prerendering."을 낸다.

설치된 가이드의 수정 예와 현재 피드가 같다. 페이지는 동기 함수이고, 프로미스를 경계 안으로 넘긴다.

```
export default function FeedPage({
  searchParams,
}: {
  searchParams: Promise<{
    page: string
    limit: string
    sort: string
    search: string
  }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <FeedContent searchParams={searchParams} />
    </Suspense>
  )
}
```

```
export default async function FeedContent({
  searchParams,
}: {
  searchParams: Promise<{
```

```
    page: string
    limit: string
    sort: string
    search: string
  }>
}) {
  const { page, limit, sort, search } = await searchParams
  return (
    <div className="flex flex-col gap-4 p-4">
      <h1>Feed Page</h1>
      <p>page: {page}</p>
      <FeedNextButton page={page} limit={limit} sort={sort} search={search} />
    </div>
  )
}
```

경계의 `fallback`은 `<p>Loading...</p>`다. 페이지 함수는 그 경계를 바로 반환하고, 문단과 버튼은 `searchParams`가 풀린 뒤 스트리밍된다.

블로그와 상품도 같은 뼈대다. 차이는 프로미스 내용이다.

라우트	프로미스	await 위치	클라이언트 자식
`/blog/name`	`{ blogId: string }`	`BlogContent`	없음
`/product/laptop/gram`	`{ productCategoryName: string[] }`	`ProductContent`	없음
`/feed?page=1&limit=10&sort=asc&search=cat`	쿼리 네 키	`FeedContent`	`FeedNextButton`

`/product/laptop/gram`의 배열은 `["laptop", "gram"]`이고, `join("/")` 결과는 `laptop/gram`이다. `console.log`는 그 서버 컴포넌트가 렌더될 때 서버 터미널에 찍힌다. 제목은 `CustomTitle`이다. `TitleType="h1"`이라 결과는 `h1`이고, 클래스는 `twMerge("text-2xl font-bold text-center text-white", "text-2xl font-bold")`다. 두 문자열 모두 소스에 완성된 클래스로 적혀 있다.

세 페이지 모두 `generateStaticParams`와 `export const instant = false`가 없다. `instant = false`는 블로킹을 허용하는 표시다. 쿼리를 읽어 주지는 않는다.

5. 버튼이 받는 값

`FeedNextButton`은 프로미스를 받지 않는다. 서버가 푼 문자열 네 개를 props로 받는다.

```
"use client"
import { useRouter } from "next/navigation"

export default function FeedNextButton({
  page,
  limit,
```

```
  sort,
  search,
}: {
  page: string
  limit: string
  sort: string
  search: string
}) {
  const router = useRouter()
  return (
    <button
      onClick={() => {
        router.push(
          `/feed?page=${parseInt(page) + 1}&limit=${limit}&sort=${sort}&search=${search}`
        )
      }}
    >
      Next
    </button>
  )
}
```

`useRouter`는 클라이언트 컴포넌트에서 경로를 바꾸는 훅이다. 서버 컴포넌트인 `FeedContent`는 이 훅을 호출하지 않는다. 훅 호출은 버튼 컴포넌트 본문 맨 위에 있다. `await` 뒤에 두지 않는다.

6. 공식 문서

- page의 searchParams. 로컬 `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/page.md`. 프로미스, 문자열, 반복 키는 배열
- cookies, headers, searchParams를 Suspense로 감싸기. 로컬 `node_modules/next/dist/docs/01-app/02-guides/migrating-to-cache-components.md`. 프로미스를 경계 안 컴포넌트로 넘기고 거기서 `await`
- blocking-prerender-runtime
- useRouter. 로컬 `node_modules/next/dist/docs/01-app/03-api-reference/04-functions/use-router.md`. `"use client"` 파일의 예
- Dynamic Routes
- React Suspense
- MDN async function
- MDN script async
- MDN parseInt
- HTML 모듈 스크립트. `type="module"`은 `defer`와 같이 문서 파싱을 막지 않고, 문서가 파싱된 뒤 순서대로 실행된다